

Corso di Laurea in Informatica A.A. 2024-2025

Laboratorio di Sistemi Operativi

Alessandra Rossi

Docker-compose

Docker Composer, un tool di gestione piuttosto versatile che rende l'adozione di Docker ancora più agevole e facilmente gestibile l'adozione di applicazioni di questo tipo.

[Docker Compose è un tool ufficiale](#) installabile su Linux (incluso Raspberry Pi OS), Windows, macOS e che consente, in modo piuttosto pratico, di gestire i propri container salvandone la configurazione di istanziamento in un unico file di configurazione in formato YAML, per esempio:

Docker-compose

Questo file di configurazione, per esempio, istruisce Docker Compose sull'instanziamento di due container, **db** (basato sull'immagine del motore MySql per Docker) e **wordpress** (basato sull'immagine del CRM [WordPress per Docker](#)) e altro

```
version: '3'
services:
  db:
    build: ./mysql
    image: poc/mysql-for-wordpress
    volumes:
      - db_data:/var/lib/mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: somewordpress
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: wordpress
  wordpress:
    depends_on:
      - db
    image: wordpress:latest
    ports:
      - "8000:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wordpress
      WORDPRESS_DB_PASSWORD: wordpress
      WORDPRESS_DB_NAME: wordpress
volumes:
  db_data: {}
```


Docker-compose

Docker Compose serve, per l'appunto, a gestire in modo immediato i propri container. I suoi comandi infatti consentono di:

- avviare, fermare, e riavviare le applicazioni istanziate;
- vedere lo stato dei servizi in esecuzione;
- consultare lo stream dei log dei servizi in esecuzione;
- eseguire comandi all'interno dei container

e altro.

In pratica, senza l'adozione di Docker Compose ogni qual volta è necessario mettere in esecuzione un container, è necessario conoscerne il comando ed eseguirlo.

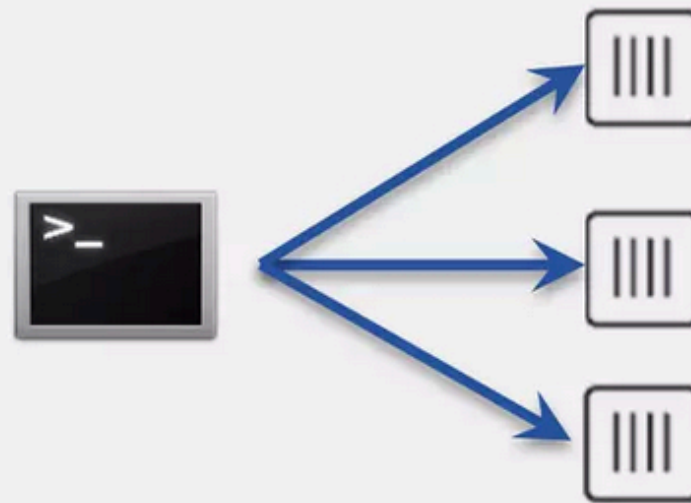
Docker-compose

Usando Docker Compose affiancato a Docker, ogni qual volta si ha necessità di aggiungere un container al proprio “stack” è sufficiente aggiungere una porzione di codice di configurazione ad esso relativa al proprio file `docker-compose.yaml`, il quale consentirà a Docker Compose di “conoscere” le caratteristiche del nuovo container (oltre a quelli già presenti) e quindi consentirne una rapida e pratica gestione.

Docker-compose

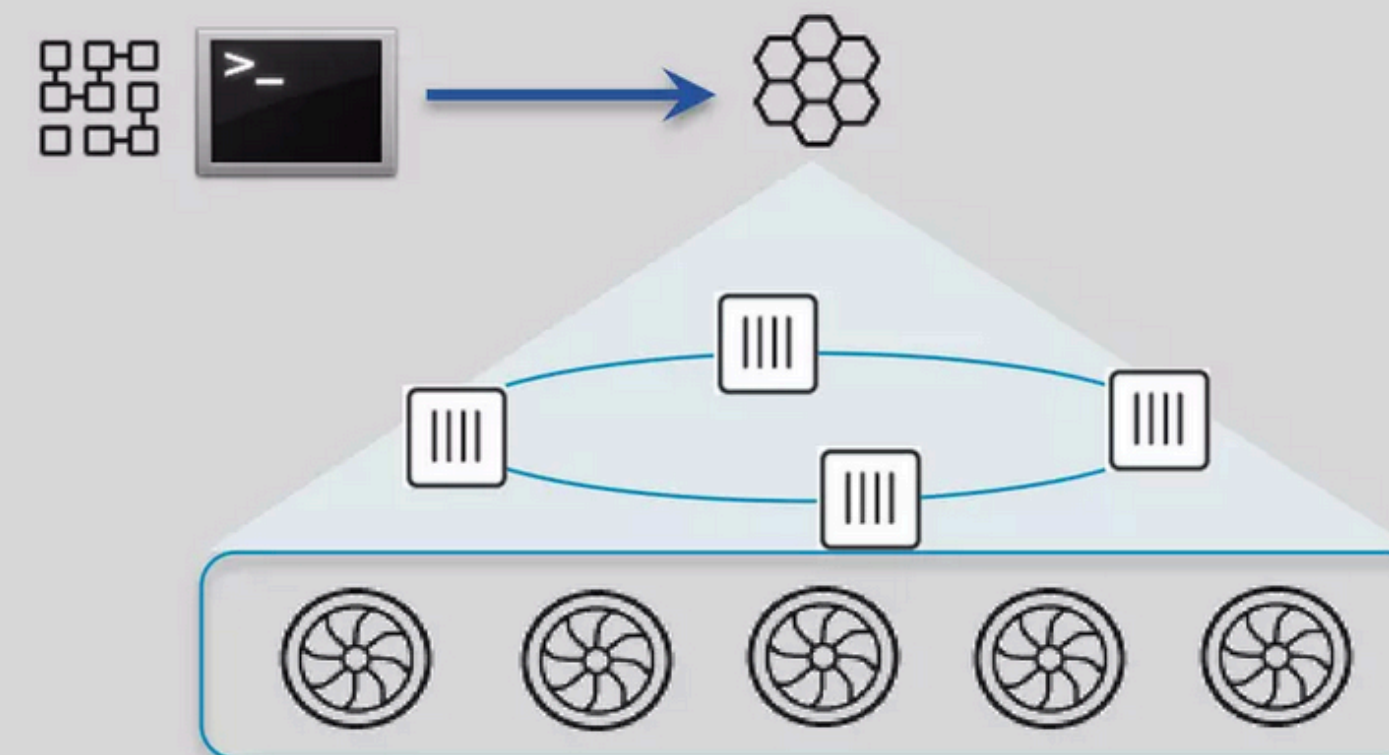
Without Compose

- Build and run one container at a time
- Manually connect containers together
- Must be careful with dependencies and start up order



With Compose

- Define multi container app in compose.yml file
- Single command to deploy entire app
- Handles container dependencies
- Works with Docker Swarm, Networking, Volumes, Universal Control Plane



Docker-compose

Per praticità.

Immaginiamo banalmente di dover amministrare **un certo numero di container**: volendo istanziare tutte le componenti necessarie in un sistema di media complessità, potremmo arrivare a definirne anche una decina.

Va da sé che per ognuno sarebbe **necessario segnarsi da parte il corretto comando di esecuzione** (in base ai tanti parametri diversi di ognuno) per eseguirlo nuovamente ogni qual volta sia necessario cancellare e ricreare il container (per esempio, in caso di aggiornamento dell'immagine per sopravvenuto aggiornamento dell'applicazione).

Con Docker Compose, invece, una volta dichiarate nel suo file di configurazione le configurazioni dei vari container, **di esse ci si può dimenticare**: basterà infatti utilizzare i suoi comandi rapidi.

Docker-compose

A tool for defining and running multi-container Docker applications

1

Compose works in all environments: production, staging, development, testing, as well as CI workflows.

3

With Compose, you use a YAML file to configure your application's services.

2

With a single command, you create and start all the services from your configuration

4

“docker-compose” o “docker compose”?

Il modo più recente per eseguire Docker Compose è quello di utilizzarlo come plug-in di Docker: in sostanza, lo si installa come plug-in e questo lo aggiunge quale possibile sotto-comando del comando principale “docker”. Precedentemente (ma la pratica è ancora valida e operativa), Docker Compose poteva venire installato **come applicazione a sé**, la quale poteva venire evocata col comando “docker-compose”.

In pratica, i comandi (esemplificativi):

```
docker compose up -d  
e  
docker-compose up -d
```

sono **negli effetti la stessa cosa**, ma possono essere utilizzati:

- il primo, solo installando Docker Compose come plug-in di Docker;
- il secondo, solo installando Docker Compose come applicazione a sé (ormai pratica deprecata).

Si possono installare entrambi, anche se ha poco senso. In caso si possono utilizzare entrambi i comandi.

<https://docs.docker.com/compose/install/linux/>

Docker-compose

<https://docs.docker.com/compose/install/>

Docker-compose

Using Compose is essentially a three-step process:

1. Define your app's environment with a `Dockerfile` so it can be reproduced anywhere.
2. Define the services that make up your app in a `compose.yaml` file so they can be run together in an isolated environment.
3. Run `docker compose up` and the Docker compose command starts and runs your entire app.

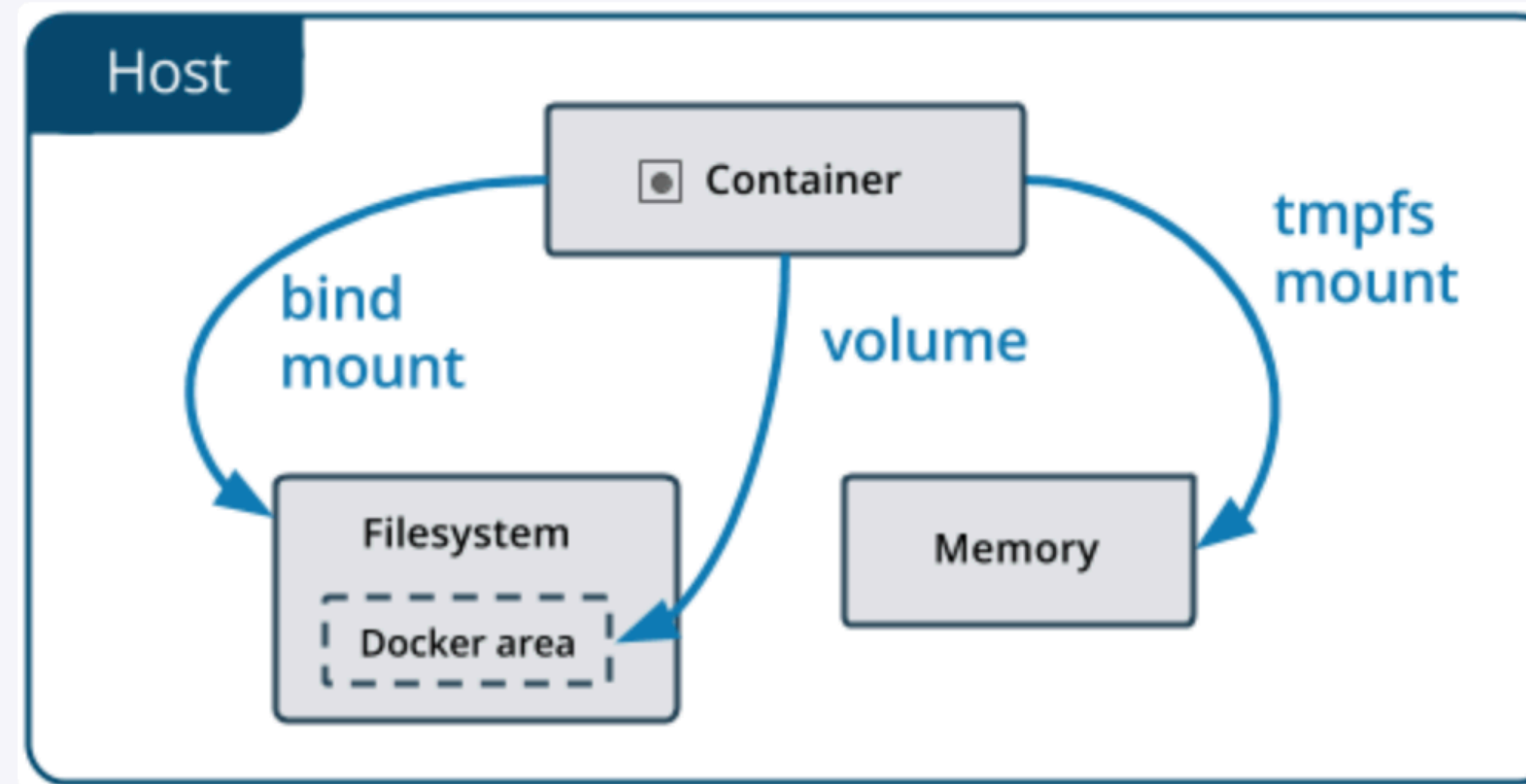
Docker-compose

By default all files created inside a container are stored on a writable container layer. This means that:

- The data doesn't persist when that container no longer exists, and it can be difficult to get the data out of the container if another process needs it.
- A container's writable layer is tightly coupled to the host machine where the container is running. You can't easily move the data somewhere else.
- Writing into a container's writable layer requires a storage driver to manage the filesystem. The storage driver provides a union filesystem, using the Linux kernel. This extra abstraction reduces performance as compared to using *data volumes*, which write directly to the host filesystem.

Docker has two options for containers to store files on the host machine, so that the files are persisted even after the container stops: volumes, and bind mounts.

Docker-compose



- Volumes are stored in a part of the host filesystem which is *managed by Docker* (`/var/lib/docker/volumes/` on Linux). Non-Docker processes should not modify this part of the filesystem. Volumes are the best way to persist data in Docker.
- Bind mounts may be stored anywhere on the host system. They may even be important system files or directories. Non-Docker processes on the Docker host or a Docker container can modify them at any time.
- `tmpfs` mounts are stored in the host system's memory only, and are never written to the host system's filesystem.

Docker-compose

File Compose: l'utilizzo di un file YAML, tipicamente denominato *docker-compose.yml*, Docker Compose consente di definire i servizi, le reti e i volumi dell'applicazione. Questo file funge da modello completo per l'intera applicazione, semplificando la gestione, la riproduzione e la condivisione della configurazione dell'applicazione.

Servizi: In Docker Compose i servizi indicano singole istanze di container create da specifiche immagini Docker. Questi servizi sono la base di un'applicazione multi-container e possono essere interconnessi attraverso le reti, condividendo i dati con i volumi.

Reti: Docker Compose facilita la creazione di reti personalizzate per le applicazioni containerizzate, consentendo una comunicazione fluida tra i servizi. Sebbene Compose generi automaticamente una singola rete per tutti i servizi per impostazione predefinita, è possibile definire reti aggiuntive per isolare servizi specifici o stabilire topologie complesse.

Volumi: Docker Compose offre volumi per conservare i dati generati e utilizzati dai contenitori Docker. I volumi facilitano la condivisione dei dati tra i contenitori e garantiscono la conservazione dei dati quando un contenitore viene rimosso o aggiornato.

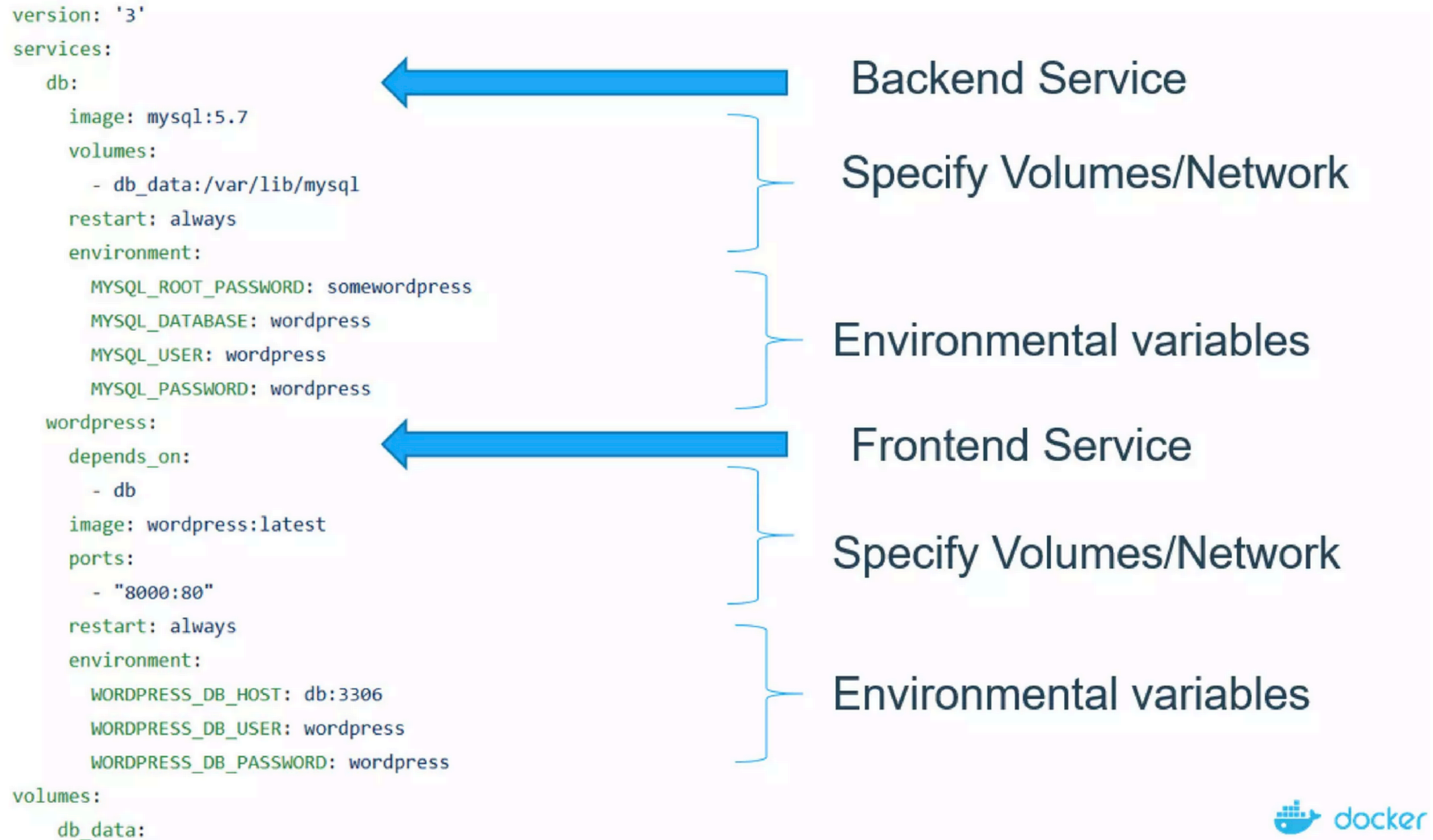
Scalabilità: Con Docker Compose è possibile scalare facilmente i servizi per gestire carichi di lavoro maggiori, regolando il numero di istanze del contenitore per un servizio specifico. Questo può essere fatto usando l'attributo `scale` o l'opzione `--scale` della riga di comando.

Sostituzione di variabili d'ambiente: Docker Compose supporta la sostituzione delle variabili d'ambiente, consentendo di adattare il file Compose a vari ambienti o fasi della pipeline di sviluppo. Questa funzione consente una maggiore flessibilità e personalizzazione in diversi scenari di distribuzione.

Docker-compose

Compose preserves all volumes used by your services. When `docker compose up` runs, if it finds any containers from previous runs, it copies the volumes from the old container to the new container. This process ensures that any data you've created in volumes isn't lost.

Esempio



Esempio

PREREQUISITI

You need to have Docker Engine and Docker Compose on your machine. You can either:

- Install [Docker Engine](#) and [Docker Compose](#) as standalone binaries
- Install [Docker Desktop](#) which includes both Docker Engine and Docker Compose

You don't need to install Python or Redis, as both are provided by Docker images.

STEP 1: DEFINIRE DIPENDENZE

- Create a directory for the project:

```
$ mkdir composetest
```

```
$ cd composetest
```


Docker-compose

- Create a file called app.py in your project directory and paste the following code in:

```
import time

import redis
from flask import Flask

app = Flask(__name__)
cache = redis.Redis(host='redis', port=6379)

def get_hit_count():
    retries = 5
    while True:
        try:
            return cache.incr('hits')
        except redis.exceptions.ConnectionError as exc:
            if retries == 0:
                raise exc
            retries -= 1
            time.sleep(0.5)

@app.route('/')
def hello():
    count = get_hit_count()
    return 'Hello World! I have been seen {} times.\n'.format(count)
```

Docker-compose

- Create a file requirements.txt in your project directory and paste the following code in:

```
flask
```

```
redis
```

Docker-compose

STEP 2: CREARE DOCKERFILE

- The Dockerfile is used to build a Docker image. The image contains all the dependencies the Python application requires, including Python itself. In your project directory, create a file named `Dockerfile` and paste the following code in

```
# syntax=docker/dockerfile:1
FROM python:3.7-alpine
WORKDIR /code
ENV FLASK_APP=app.py
ENV FLASK_RUN_HOST=0.0.0.0
RUN apk add --no-cache gcc musl-dev linux-headers
COPY requirements.txt requirements.txt
RUN pip install -r requirements.txt
EXPOSE 5000
COPY . .
CMD ["flask", "run"]
```


Docker-compose

STEP 3: DEFINE SERVICES

- Create a file called compose.yaml in your project directory and paste the following:

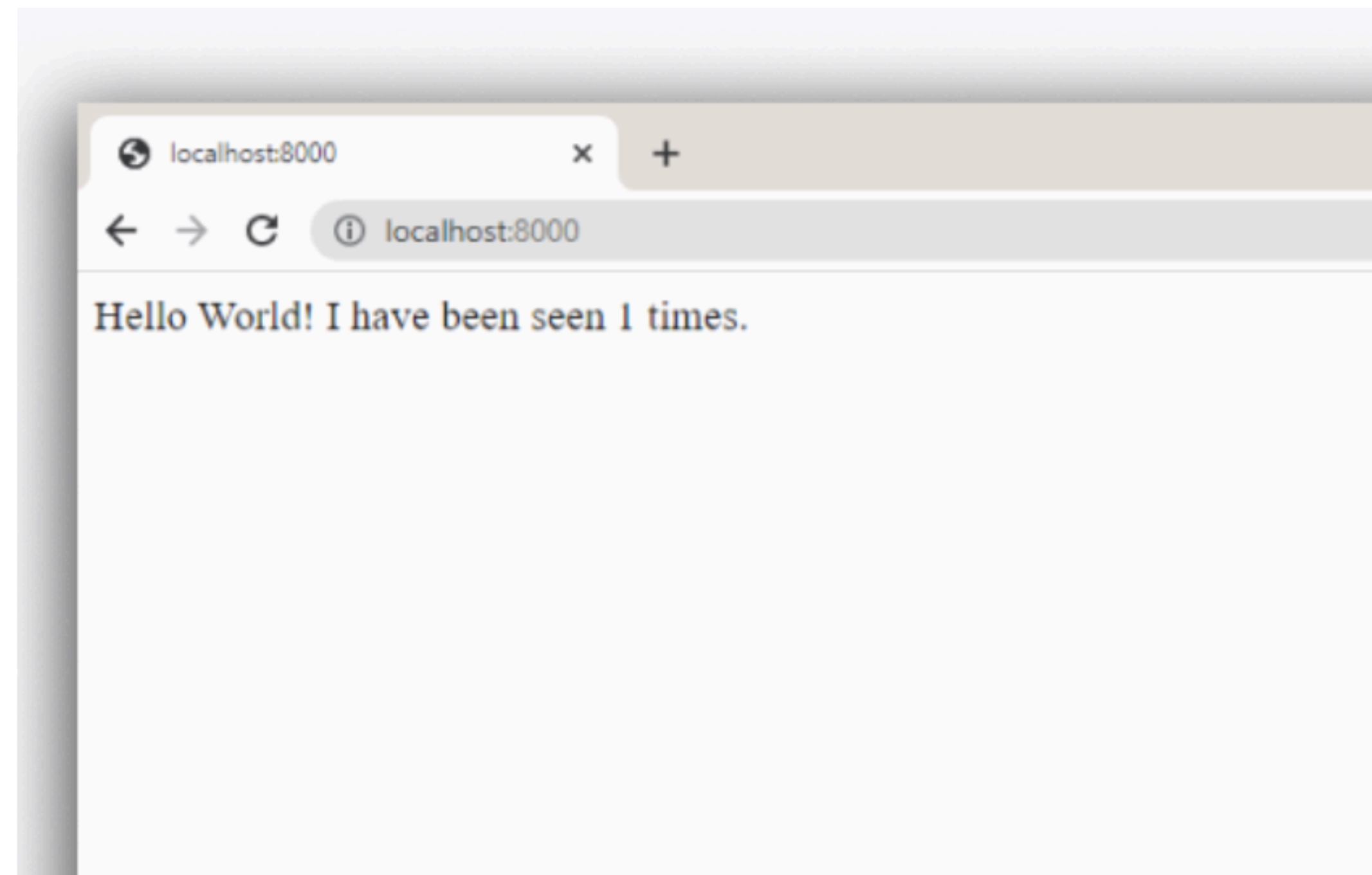
```
services:
  web:
    build: .
    ports:
      - "8000:5000"
  redis:
    image: "redis:alpine"
```


Docker-compose

STEP 4: BUILD e RUN

- From your project directory, start up your application by running `docker compose up`
- Enter `http://localhost:8000/` in a browser to see the application running.

If this doesn't resolve, you can also try `http://127.0.0.1:8000`. You should see a message in your browser saying:



Docker-compose

STEP 5: AGGIUNGI BIND MOUNT

```
services:
  web:
    build: .
    ports:
      - "8000:5000"
    volumes:
      - .:/code
    environment:
      FLASK_DEBUG: "true"
  redis:
    image: "redis:alpine"
```



Università degli Studi di Napoli Federico II

THANK YOU FOR YOUR ATTENTION

TRUST ME ...
I AM A ROBOT!

